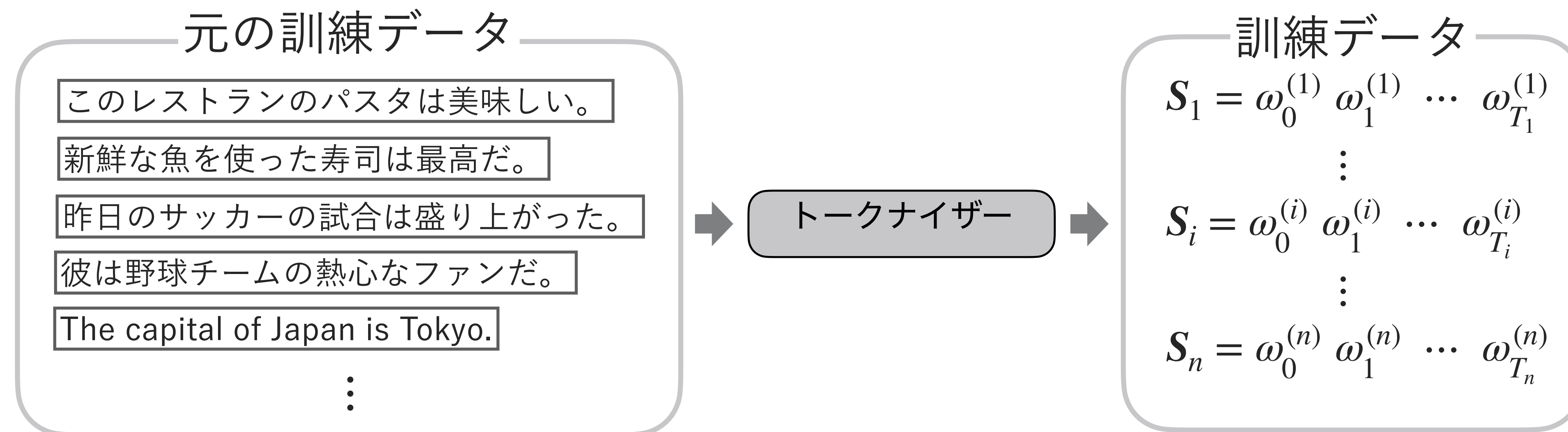


生成タスクへの自己回帰型アプローチ

生成タスク > 自己回帰型の生成 > モデル

系列データの生成タスク学習

- 訓練データとしては、ラベル無しのトークン列が得られているとする



- ここから、新規のトークン列を生成できる（条件付き）生成器を学習したい

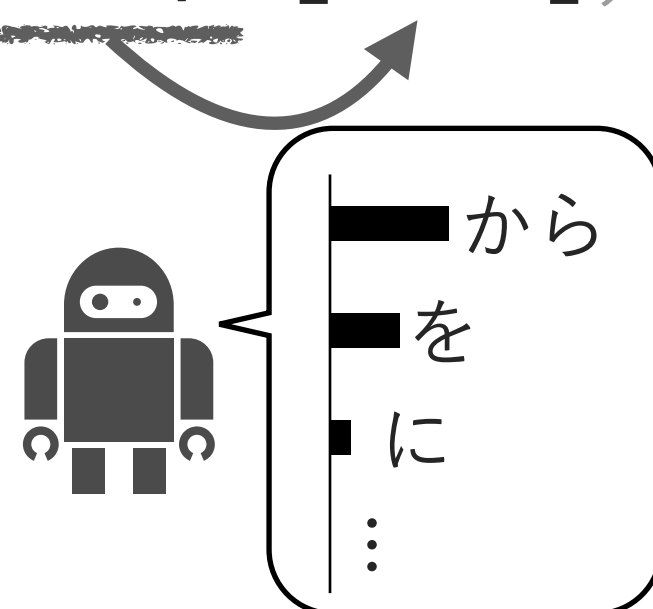
生成タスク > 自己回帰型の生成 > モデル

自然言語処理では，このような条件付き確率モデルは言語モデルと呼ばれる

■ 言語モデル (language model)

- 途中までのトークン列から，次のトークンの生成確率を予測するモデルのこと
- 自然言語処理で古くから扱われてきたが，これをニューラルネットでアテンションを中心に使うアーキテクチャで大規模化したものがGPT等

途中までの文章 [?]，次のトークンの確率を予測

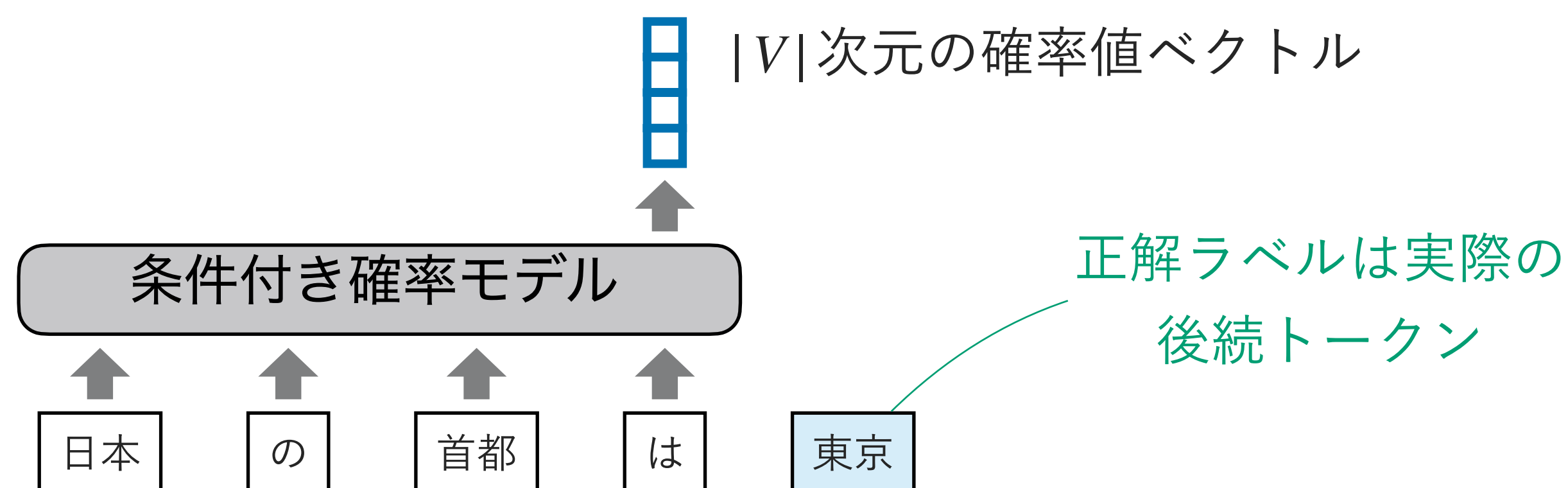


生成タスク > 自己回帰型の生成 > 学習

学習は、次トークン予測という確率論的分類の学習によって行う

■ 次トークン予測 (next token prediction)

- 訓練データは $\{S_i\}_{i=1}^n$ (ここで、各 S_i は、トークン列 $S_i = \omega_0^{(i)} \omega_1^{(i)} \dots \omega_{T_i}^{(i)}$)
- 最尤法ベースで、多クラスの確率論的分類の学習を行う



生成タスク > 自己回帰型の生成 > 学習

次トークン予測による学習

- つまり、基本の目的関数は

$$L(\theta) := \sum_{i=1}^n \sum_{t=1}^{T_i} -\log g_{\theta}(\omega_0^{(i)}, \dots, \omega_{t-1}^{(i)})[\omega_t^{(i)}]$$

- 実際には、正則化なども付きつつ、これを確率的勾配降下法で最適化する
- なお通常 $g_{\theta}(\omega_0, \dots, \omega_{t-1}) = \text{Softmax}(s_{\theta}(\omega_0, \dots, \omega_{t-1}))$ という形でモデル化される

生成タスク > 自己回帰型の生成 > 生成

学習したモデルは、自己回帰的な生成に使う

- 学習は、条件付き確率を近似するように行ったが、最終目的は近似ではない

$$g_{\theta}(\omega_0 \cdots \omega_{t-1}) \simeq \left(p(\omega \mid \omega_0 \cdots \omega_{t-1}) \right)_{\omega \in V}$$

- タスクは生成なので、新データの生成を行いたい

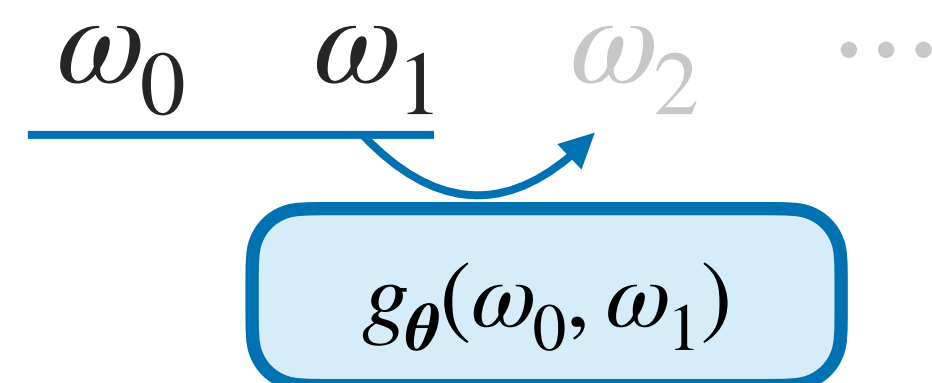


$\omega_0 \cdots \omega_T$

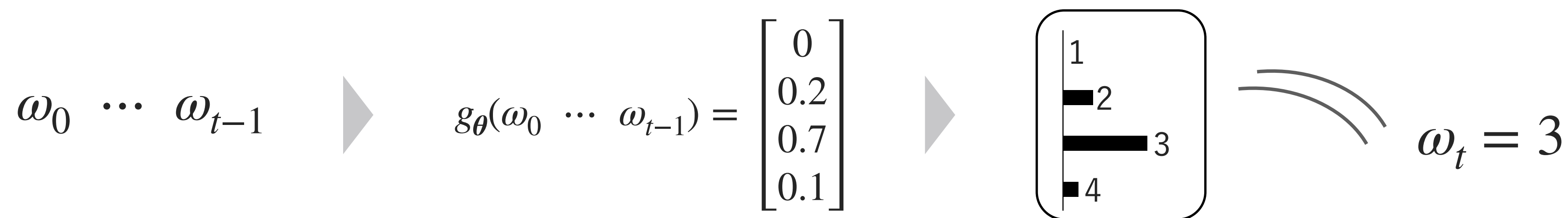
生成タスク > 自己回帰型の生成 > 生成

学習したモデルは、自己回帰的な生成に使う

- 学習済みの g_θ を用いて「すでに生成した部分から次トークンを生成」を繰り返す



- 1回ずつのサンプリングは、語彙は有限なので、連続的データほど難しくくない



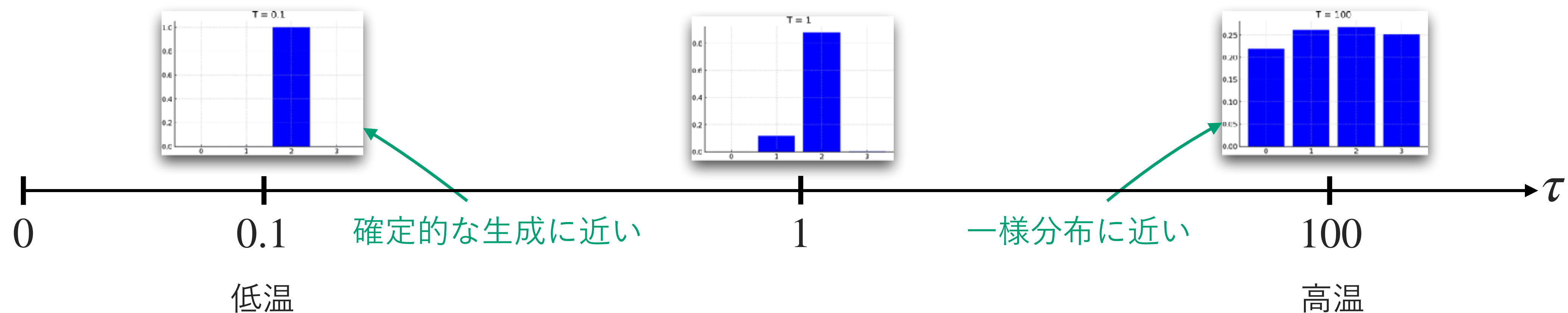
- しかし、いくつかの工夫は用いられる

生成タスク > 自己回帰型の生成 > 温度パラメーター

温度パラメーターを上げ下げして，生成結果の多様性をコントロールする

$$\text{Softmax}_\tau(\mathbf{z}) = \left(\frac{\exp\left(\frac{z_\omega}{\tau}\right)}{\sum_{v \in V} \exp\left(\frac{z_v}{\tau}\right)} \right)_{\omega \in V}$$

- ランダム性を減らしたい場合は τ を下げ，意外な出力を得たい場合は τ を上げる



生成タスク > 自己回帰型の生成 > Top-p生成

最も確率が高いトークンではなく、ある程度確率が高いトークンを生成

■ 貪欲法 (greedy sampling)

- 生成の各ステップで最も確率が高いトークンを選択するという生成方式
- ありきたりな文ばかりが生成されてしまう傾向がある

■ Top-p生成 (nucleus sampling)

- 生成の各ステップで、確率の高い順に、確率の合計が p になるまで生成候補に追加し、その中で確率の重みに従ってランダムにトークンを選択する
- LLMでの主要な生成方式となっている (p は 0.9 や 0.95 などが用いられる)

【用語】 nucleus は、核という意味。ここでは、確率の高いトークンの候補群のことが語彙の核と呼ばれている。【補足】 Nucleus samplingはHoltzman et al. (2019)により導入された。系列生成の方法には他にビームサーチ (beam search) などもあるが、Holtzman et al. (2019) はビームサーチを使用すると面白みがなく繰り返しの多い「退化した」テキストが生成されがち、という経験的問題を指摘し、対案としてnucleus samplingを導入した。【参考】 Holtzman et al. (2019). The curious case of neural text degeneration. International Conference on Learning Representations. <https://openreview.net/forum?id=rygGQyrFvH>

生成タスク > 自己回帰型の生成 > 補足

自己回帰型のアプローチはどこから来たのか？

- トークン列の発生確率 $p(\omega_0 \cdots \omega_T)$ は、次のように因子分解できる

$$p(\omega_0 \cdots \omega_T) = p(\omega_T \mid \omega_0 \cdots \omega_{T-1}) \cdots p(\omega_2 \mid \omega_0 \omega_1) \cdot p(\omega_1 \mid \omega_0) \cdot p(\omega_0)$$

- 確率の連鎖律： $p(x, y, z) = p(x, y \mid z)p(z) = p(x \mid y, z)p(y \mid z)p(z)$ の繰り返し

- そこで、この因子分解の要素を各々近似すれば全体の確率のモデルを構成できる

$$g_{\theta}(\omega_0 \cdots \omega_{t-1}) \simeq \left(p(\omega \mid \omega_0 \cdots \omega_{t-1}) \right)_{\omega \in V}$$

- 任意の長さの系列を入力できる g_{θ} ならば、これを単一のモデルで学習できる

【補足】 ここで、 V はトークン語彙の集合。引き続き、生成対象のトークンの候補（語彙）は有限個であると仮定する。それにより、語彙の条件付き確率を決めさえすれば、そこからのサンプル生成は容易だとする。

生成タスク > 自己回帰型の生成



系列データの生成で用いられる，自己回帰型のアプローチの生成を解説しました

- 任意の長さの系列を入力できるモデルとして，トランスフォーマー系のモデルが活躍している
 - 大規模なパラメーターで，大規模なデータで，大量の計算コストを投下して，ここで解説した仕組みで作られた生成モデルが，LLMと呼ばれている
 - （ここで解説した学習は，LLMの素体を作る事前学習フェーズの学習法）
- その仕組みは，自己回帰的な生成であり，そのために条件付き確率の近似を行うので，確率論的分類と同じ学習技法が使われている